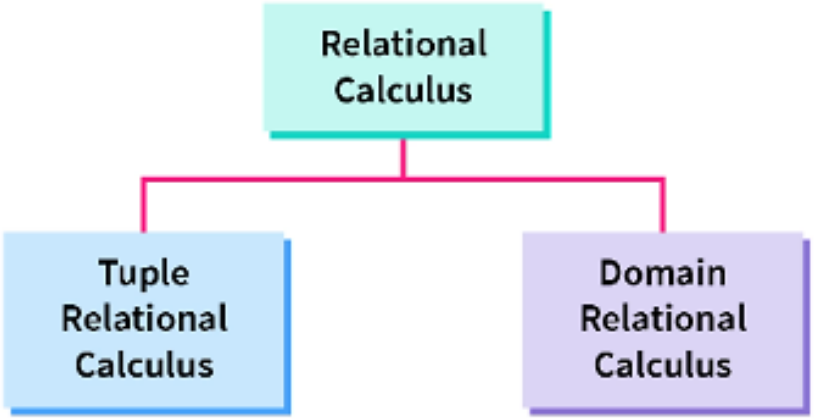
# RELATIONAL CALCULUS

# INTRODUCTION

- Procedural Language - those language which clearly define how to get the required results from the database.
  - Ex. Relational Algebra
- Declarative Language - those language that only cares about what to get from the database without getting into how to get the results.
  - Ex. Relational Calculus

# DEFINITION

- Relational calculus is a non-procedural query language, and instead of algebra, it uses mathematical predicate calculus.
- When applied to databases, it is found in two forms :
  - Tuple relational calculus which was originally proposed by Codd in the year 1972
  - Domain relational calculus which was proposed by Lacroix and Pirotte in the year 1977



# TUPLE RELATIONAL CALCULUS

- In the tuple relational calculus, you will have to find tuples for which a predicate is true.
- Tuple calculus provides only the description of the query but it does not provide the methods to solve it
- The calculus is dependent on the use of tuple variables.
  - A tuple variable is a variable that 'ranges over' a named relation
  - i.e., a variable whose only permitted values are tuples of the relation.

# TUPLE RELATIONAL CALCULUS

A query is expressed as

$$\{t \mid P(t)\}$$

where

- t = resulting tuples
- P(t) = known as Predicate and these are the conditions that are used to fetch t

Thus, it generates set of all tuples t, such that Predicate P(t) is true for t.

# TUPLE RELATIONAL CALCULUS

1. A tuple relational calculus expression is of the form : { t | P(t) } where P is a formula. Several tuple variables may appear in a formula.
2. A tuple variable is said to be a free variable unless it is quantified by a ∃ or a ∀. Then it is said to be a bound variable.
3. A formula is built of atoms. An atom is one of the following forms:
   - s ∈ r where s is a tuple variable, and r is a relation (∉ is not allowed)
   - s[x] Θ u[y] where s and u are tuple variables, and x and y are attributes, and Θ is a comparison operator ( <, ≤, =, ≠, >, ≥ )
   - s[x] Θ c, where c is a constant in the domain of attribute x

# TUPLE RELATIONAL CALCULUS

4. Formulae are built up from atoms using the following rules:
   - An atom is a formula.
   - If P is a formula, then so are P and (P).
   - If $P_1$ and $P_2$ are formulae, then so are $P_1 \vee P_2$ , $P_1 \wedge P_2$ , and $P_1 \Rightarrow P_2$ .
   - If P(s) is a formula containing a free tuple variable s, then $\exists s \in r(P(s))$ and $\forall s \in r(P(s))$ are formulae also.

5. Note some equivalences:
   - $P_1 \wedge P_2 = \sim(\sim P_1 \vee \sim P_2)$
   - $\forall t \in r(P(t)) = \sim\exists t \in r(\sim P(t))$
   - $P_1 \Rightarrow P_2 = \sim P_1 \vee P_2$

# TUPLE RELATIONAL CALCULUS

- P(t) may have various conditions logically combined with
  - OR (∨),
  - AND (∧),
  - NOT(¬).
- It also uses quantifiers:
  - ∃ t ∈ r (Q(t)) = ”there exists” a tuple in t in relation r such that predicate Q(t) is true.
  - ∀ t ∈ r (Q(t)) = Q(t) is true “for all” tuples in relation r.

## Table: CUSTOMER

| Customer name | Street | City |
|---|---|---|
| Saurabh | A7 | Patiala |
| Mehak | B6 | Jalandhar |
| Sumiti | D9 | Ludhiana |
| Ria | A5 | Patiala |

## Table: ACCOUNT

| Account number | Branch name | Balance |
|---|---|---|
| 1111 | ABC | 50000 |
| 1112 | DEF | 10000 |
| 1113 | GHI | 9000 |
| 1114 | ABC | 7000 |

## Table: BORROWER

| Customer name | Loan number |
|---|---|
| Saurabh | L33 |
| Mehak | L49 |
| Ria | L98 |

## Table: BRANCH

| Branch name | Branch city |
|---|---|
| ABC | Patiala |
| DEF | Ludhiana |
| GHI | Jalandhar |

## Table: LOAN

| Loan number | Branch name | Amount |
|---|---|---|
| L33 | ABC | 10000 |
| L35 | DEF | 15000 |
| L49 | GHI | 9000 |
| L98 | DEF | 65000 |

## Table: DEPOSITOR

| Customer name | Account number |
|---|---|
| Saurabh | 1111 |
| Mehak | 1113 |
| Sumiti | 1114 |

# TUPLE RELATIONAL CALCULUS

Find the loan number, branch, amount of loans of greater than or equal to 10000 amount.

$$\{t \mid t \in LOAN \wedge t[amount] >= 10000\}$$

$$\{t \mid LOAN\ (t) \wedge t.amount >= 10000\}$$

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L33 | ABC | 10000 |
| L35 | DEF | 15000 |
| L49 | GHI | 9000 |
| L98 | DEF | 65000 |

| Loan number | Branch name | Amount |
|---|---|---|
| L33 | ABC | 10000 |
| L35 | DEF | 15000 |
| L98 | DEF | 65000 |

# TUPLE RELATIONAL CALCULUS

Find the loan number for each loan of an amount greater or equal to 10000.

```
{t| ∃ s ∈ loan(t[loan number] =
s[loan number] ∧ s[amount]>=10000)}
```

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L33 | ABC | 10000 |
| L35 | DEF | 15000 |
| L49 | GHI | 9000 |
| L98 | DEF | 65000 |

| Loan number |
|---|
| L33 |
| L35 |
| L98 |

# TUPLE RELATIONAL CALCULUS

Find the names of all customers who have a loan and an account at the bank. Tables

**Table: BORROWER**

| Customer name | Loan number |
|---|---|
| Saurabh | L33 |
| Mehak | L49 |
| Ria | L98 |

**Table: DEPOSITOR**

| Customer name | Account number |
|---|---|
| Saurabh | 1111 |
| Mehak | 1113 |
| Sumiti | 1114 |

```
{t | ∃ s ∈ BORROWER( t[customer-name] = s[customer-name])
∧ ∃ u ∈ DEPOSITOR( t[customer-name] = u[customer-name])}
```

| Customer name |
|---|
| Saurabh |
| Mehak |

# TUPLE RELATIONAL CALCULUS

Find the names of all customers having a loan at the “ABC” branch. Tables

**Table: BORROWER**

| Customer name | Loan number |
|---|---|
| Saurabh | L33 |
| Mehak | L49 |
| Ria | L98 |

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L33 | ABC | 10000 |
| L35 | DEF | 15000 |
| L49 | GHI | 9000 |
| L98 | DEF | 65000 |

```
{t | ∃ s ∈ BORROWER(t[customer-name] = s[customer-name] ∧
∃ u ∈ LOAN(u[branch-name] = “ABC” ∧ u[loan-number] =
s[loan-number]))}
```

| Customer name |
|---|
| Saurabh |

# DOMAIN RELATIONAL CALCULUS

- In Domain Relational Calculus, a query is expressed as,

$$\{< x_1, x_2, x_3, \ldots, x_n > \mid P(x_1, x_2, x_3, \ldots, x_n)\}$$

  where, $< x_1, x_2, x_3, \ldots, x_n >$ represents resulting domains variables and $P(x_1, x_2, x_3, \ldots, x_n)$ represents the condition or formula equivalent to the Predicate calculus.

- Predicate Calculus Formula:
  - Set of all comparison operators
  - Set of connectives like and, or, not
  - Set of quantifiers

## Table: CUSTOMER

| Customer name | Street | City |
|---|---|---|
| Debomit | Kadamtala | Alipurduar |
| Sayantan | Udaypur | Balurghat |
| Soumya | Nutanchati | Bankura |
| Ritu | Juhu | Mumbai |

## Table: LOAN

| Loan number | Branch name | Amount |
|---|---|---|
| L01 | Main | 200 |
| L03 | Main | 150 |
| L10 | Sub | 90 |
| L08 | Main | 60 |

## Table: BORROWER

| Customer name | Loan number |
|---|---|
| Ritu | L01 |
| Debomit | L08 |
| Soumya | L03 |

# DOMAIN RELATIONAL CALCULUS

Find the loan number, branch, amount of loans of greater than or equal to 100 amount.

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L01 | Main | 200 |
| L03 | Main | 150 |
| L10 | Sub | 90 |
| L08 | Main | 60 |

$$\{\langle l, b, a\rangle \mid \langle l, b, a\rangle \in LOAN \wedge (a \geq 100)\}$$

| Loan number | Branch name | Amount |
|---|---|---|
| L01 | Main | 200 |
| L03 | Main | 150 |

# DOMAIN RELATIONAL CALCULUS

Find the loan number for each loan of an amount greater or equal to 150.

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L01 | Main | 200 |
| L03 | Main | 150 |
| L10 | Sub | 90 |
| L08 | Main | 60 |

$$\{\langle l \rangle \mid \exists\, b, a\ (\langle l, b, a \rangle \in LOAN \wedge (a \geq 150)\}$$

| Loan number |
|---|
| L01 |
| L03 |

# DOMAIN RELATIONAL CALCULUS

Find the names of all customers having a loan at the "Main" branch and find the loan amount .

**Table: LOAN**

| Loan number | Branch name | Amount |
|---|---|---|
| L01 | Main | 200 |
| L03 | Main | 150 |
| L10 | Sub | 90 |
| L08 | Main | 60 |

**Table: BORROWER**

| Customer name | Loan number |
|---|---|
| Ritu | L01 |
| Debomit | L08 |
| Soumya | L03 |

| Customer Name | Amount |
|---|---|
| Ritu | 200 |
| Debomit | 60 |
| Soumya | 150 |

$$\{\langle c, a\rangle \mid \exists\, l\, (\langle c, l\rangle \in \text{BORROWER} \wedge \exists\, b\, (\langle l, b, a\rangle \in \text{LOAN} \wedge (b = \text{“Main”})))\}$$

# RELATIONAL ALGEBRA VS CALCULUS

| Sr. No. | Key | Relational Algebra | Relational Calculus |
|---|---|---|---|
| 1 | Language Type | Relational Algebra is procedural query language. | Relational Calculus is a non-procedural or declarative query language. |
| 2 | Objective | Relational Algebra targets how to obtain the result. | Relational Calculus targets what result to obtain. |
| 3 | Order | Relational Algebra specifies the order in which operations are to be performed. | Relational Calculus specifies no such order of executions for its operations. |
| 4 | Dependency | Relational Algebra is domain independent. | Relational Calculus can be domain dependent. |
| 5 | Programming Language | Relational Algebra is close to programming language concepts. | Relational Calculus is not related to programming language concepts. |